

Sentinelleai — Audit Report

ERC20Token.sol · c877a284-417f-4122-a8b7-dd76f20fe9bd · 2026-05-22 16 h 40 min 50 s

CAUTION 58 / 100 4 consolidated findings

The ERC20Token contract presents a moderate risk profile. The most significant confirmed finding (SC02) is the use of `balanceOf(address(this))` in `withdrawFees()`, which is deterministically flagged by the static sentinel and confirmed by developer and historian agents: an attacker can directly transfer tokens to the contract to inflate the withdrawable fee balance, bypassing the intended tax accounting. A LOW-severity zero-address validation gap (SC01) in `setMarketingWallet()` is similarly confirmed across multiple agents and the static sentinel. The historian agent raises a MEDIUM-severity concern about centralized single-EOA ownership controlling mint, pause, blacklist, and tax functions without multisig or timelock — a recognized pattern in historical exploits — though this is partially mitigated by Ownable2Step's two-step transfer mechanism and is consistent with the documented design intent for a customizable token. Three formalist-only findings (tax fee cap, blacklist check, tax logic) are informational confirmations of correct implementation rather than vulnerabilities and are rejected as false positives. Overall, the contract triggers OWASP SC01 (Broken Access Control) and SC02 (Price Oracle / Accounting Manipulation) categories and should not be deployed without addressing the `balanceOf` accounting issue and considering governance hardening.

Severity breakdown

1 HIGH 1 MEDIUM 1 LOW 1 INFO

Static sentinel pre-scan

Static sentinels matched 2 pattern(s) — 1 HIGH, 1 LOW.

HIGH SC02 · ACC-1 · line 158

balanceOf(this) used in pricing / rate calculation

Using token.balanceOf(address(this)) as an input to swap/redeem/getRate is bypassable: an attacker can ERC20-transfer tokens directly to the contract (skipping deposit) and skew the calculation. Track an internal accounting variable instead.

```
uint256 balance = balanceOf(address(this));
```

LOW SC01 · ZAD-1 · line 149

Setter accepts zero-address (no validation)

set*(address _x) functions that write the parameter directly to state can brick the protocol if called with address(0). Add require(_x != address(0)).

```
function setMarketingWallet(address newWallet) external onlyOwner { marketingWallet = newWallet; emit MarketingWalletUpdated(newWallet); }
```

Consolidated findings

HIGH CVSS 7.5 SC02 — Price Oracle / Accounting Manipulation

balanceOf(address(this)) Used for Fee Withdrawal Amount (SC02)

The `withdrawFees()` function determines the withdrawable amount using `balanceOf(address(this))`. Any party can directly ERC20-transfer tokens to the contract address, inflating the perceived fee balance without going through the tax mechanism. This allows an attacker (or the owner) to drain tokens that were not legitimately collected as fees. This matches the Venus Protocol (\$2M+, 2026-03) root cause pattern and is deterministically confirmed by the static sentinel.

```
withdrawFees(), line 158: uint256 balance = balanceOf(address(this));
```

→ Introduce a dedicated internal accounting variable (e.g., `uint256 public accumulatedFees`) that is incremented only inside the `_update()` tax collection path. Use this variable in `withdrawFees()` instead of `balanceOf(address(this))` to prevent external inflation of the withdrawable amount.

Confirmed by: [static-sentinel](#), [developer](#), [historian](#), [formalist](#)

MEDIUM CVSS 5.3 SC01 — Broken Access Control

Single-EOA Owner Controls All Privileged Functions Without Multisig or Timelock (SC01)

All critical functions — `mint`, `pause`, `blacklist`, `setTaxFee`, `setMarketingWallet`, `updateMetadata`, and `enableRevoke*` — are gated solely by the `Ownable2Step` owner. While `Ownable2Step` adds a two-step ownership transfer, there is no multisig requirement, timelock, or role separation. A compromised or malicious owner can mint unlimited tokens (until revoked), blacklist arbitrary addresses, set a 25% tax fee, and redirect fees. This matches patterns seen in Wasabi Protocol and Drift Protocol exploits.

```
All onlyOwner functions: mint(), pause(), blacklist(), setTaxFee(),
setMarketingWallet(), updateMetadata(), enableRevoke*()
```

→ Consider replacing the single-EOA owner with a 3-of-5 multisig wallet and adding a 48-hour timelock on sensitive operations such as `mint`, `setTaxFee`, and `blacklist`. Separate roles for treasury management, pausing, and minting would further reduce blast radius of a key compromise.

Confirmed by: [historian](#)

LOW CVSS 3.1 SC01 – Broken Access Control / Input Validation

setMarketingWallet Accepts Zero Address Without Validation (SC01)

The setMarketingWallet() function writes the provided address directly to state without validating against address(0). While withdrawFees() has a fallback to owner() when marketingWallet is zero, explicitly setting it to zero is likely an operator error and should be rejected to prevent confusion and potential misrouting of funds.

```
setMarketingWallet(), line 149: marketingWallet = newWallet;
```

→ Add require(newWallet != address(0), "ERC20Token: Invalid marketing wallet address"); at the start of setMarketingWallet().

Confirmed by: static-sentinel, developer, historian, formalist

INFO CVSS 0.0 Informational

Tax Fee Cap, Blacklist Check, and Tax Logic Are Correctly Implemented

The formalist agent flagged the 25% tax fee cap (require(newFee <= 2500)), the blacklist check in _update(), and the tax logic in _update() as MEDIUM findings. Review confirms these are correctly implemented per the documented design intent: the fee cap is enforced, blacklist checks prevent transfers to/from blacklisted addresses, and tax is correctly excluded for owner and internal transfers. These are not vulnerabilities.

```
_update() lines 184, 221, 226; setTaxFee() line 140
```

→ No action required. These are working-as-intended features.

Confirmed by: formalist

Historical precedents

Retrieved from the local bug-bounty corpus (~20 700 findings) by vulnerability-class match.

CRITICAL **oracle** ethereum 2021-10-27 **\$130 000 000**

CreamFinance — Price Manipulation (2021-10-27)

<https://medium.com/immunefi/hack-analysis-cream-finance-oct-2021-fc222d913fc5>

CRITICAL **oracle** arbitrum 2025-07-09 **\$41 000 000**

GMX — Share price manipulation (2025-07-09)

https://x.com/GMX_IO/status/1943336664102756471

CRITICAL **oracle** polygon 2021-11-30 **\$31 000 000**

MonoX Finance — Price Manipulation (2021-11-30)

<https://slowmist.medium.com/detailed-analysis-of-the-31-million-monox-protocol-hack-574d8c44a9c8>

HIGH **donation** zksync 2025-02-27

Venus_ZKSync — Donation Attack (2025-02-27)

<https://explorer.zksync.io/>

[tx/0x35a0172fb6bd450ceb29aa67dc85221826dfd0b7528375400b4ccf15c1eed0d8](https://explorer.zksync.io/tx/0x35a0172fb6bd450ceb29aa67dc85221826dfd0b7528375400b4ccf15c1eed0d8)

CRITICAL **access-control** ethereum 2025-05-28 **\$12 000 000**

Corkprotocol — Access Control (2025-05-28)

https://x.com/SlowMist_Team/status/1928100756156194955

HIGH **access-control** bsc 2023-03-28 **\$8 900 000**

SafeMoon Hack — Access Control (2023-03-28)

https://twitter.com/zokyo_io/status/1641014520041840640

HIGH **access-control** linea 2024-06-01 **\$6 880 000**

VeloCore — lack-of-access-control (2024-06-01)

<https://x.com/BeosinAlert/status/1797247874528645333>

MEDIUM **bm25** 2023-09-01

ondo — Incorrect calculation of totalSupply(), balanceOf() in rUSDY.sol if the rate is unlinked from \$1

<https://github.com/code-423n4/2023-09-ondo-findings/issues/481>

MEDIUM **bm25** 2024-03-01

revert-lend — Uniswap oracle should not be used as a source of pricing on L2s

<https://github.com/code-423n4/2024-03-revert-lend-findings/issues/101>

GAS **bm25** 2021-11-01

nested — Add zero-address checkers

<https://github.com/code-423n4/2021-11-nested-findings/issues/108>

MEDIUM bm25 2024-07-01

loopfi — Incorrect Supply Rate Calculation in CDPVault

<https://github.com/code-423n4/2024-07-loopfi-findings/issues/352>

HIGH bm25 2024-04-01

renzo — Wrong calculation of `totalWithdrawalQueueValue` in
`RestakeManager.calculateTVLs()` breaks deposits, withdrawals calculation of EzETH rate
calculation

<https://github.com/code-423n4/2024-04-renzo-findings/issues/156>

Generated by Sentinelleai on 2026-05-22T21:50:18.937Z. Audit ID: c877a284-417f-4122-a8b7-dd76f20fe9bd.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.